UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA



Facultad de Ciencias Químicas e Ingeniería

Ingeniero en computación

PROYECTO DE CARRERA

Taller No. 5

PROFESOR:

ALUMNA: Gutierrez Sanchez Valeria Karey 1282379

GRUPO 581

FECHA ENTREGA: 16 de septiembre de 2026

DashGuard: Prueba de concepto

1. Objetivo y alcance 3

2. Arquitectura y conexiones 3

3. Implementación mínima y criterios de cumplimiento 4

3.1. Procesamiento y formato 5

3.2. Estructura de datos 5

4. Pruebas y resultados 6

4.2 Prueba de almacenamiento y consulta de mediciones de aceleración con coordenadas GPS. 7

4.3 Prueba de registro y visualización de un recorrido mediante el módulo GPS NEO-6M. 8

4.4. Problemas encontrados y correcciones 9

5. Cronograma de las semanas restantes 9

6. Presupuesto 10

7. Matriz de riesgos y contingencias 10

8. Evaluación de viabilidad y pendientes 11

9. Actualización: GPS y registro de recorridos 11

9.1 Integración embebida y transmisión 11

9.2 Registro y consulta de recorridos 11

DashGuard: Prueba de concepto

Estado general: La arquitectura mínima ya está integrada y funciona; todavía faltan por documentar algunas evidencias y pruebas.

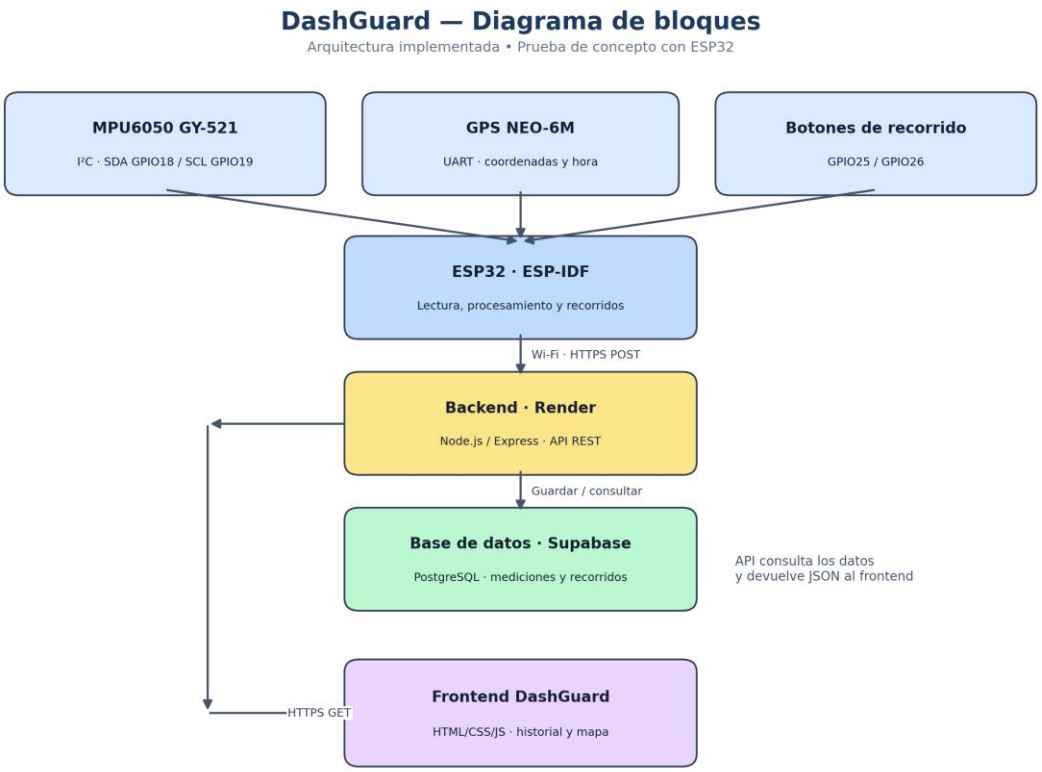
1. Objetivo y alcance

El objetivo de este taller es demostrar que el proyecto es viable: que se puede medir la aceleración con un sensor físico, procesar el dato, enviarlo por Internet a un servidor en la nube, guardarlo en una base de datos y consultarlo desde una página web. Para esta prueba se usa de forma temporal un ESP32 con un sensor MPU6050 (acelerómetro) y un módulo GPS NEO-6M. El prototipo final se construirá sobre una Raspberry Pi Zero 2 W, junto con una cámara, el GPS NEO-6M, una tarjeta microSD y una fuente de alimentación autónoma.

El dato que se envía es la magnitud de la aceleración, medida en m/s². Las filas que se muestran en pantalla son mediciones reales, no choques ya clasificados. En esta prueba reducida no se incluyen la grabación de video, la asociación de clips ni la detección definitiva de incidentes. El GPS ya entregó coordenadas reales y quedó integrado al flujo de datos; los recorridos se explican en la sección 9.

2. Arquitectura y conexiones

Flujo implementado:



Elemento	Conexión o interfaz
MPU6050 GY-521	Comunicación I²C con el ESP32 (SDA en GPIO18 y SCL en GPIO19); la alimentación y la tierra se conectan según el módulo
ESP32 → backend	Conexión Wi-Fi y envío mediante HTTPS POST a /mediciones; el certificado raíz se valida con el paquete de certificados de mbedTLS
Backend → Supabase	El servidor se conecta con credenciales guardadas como variables de entorno (no escritas en el código)
Frontend → backend	Solicitud HTTPS GET a /mediciones; la página consulta los datos cada 15 s

Backend público: https://dashguard-backend.onrender.com

Repositorio backend: https://github.com/vkgtzs/dashguard-backend

Frontend: funciona y está conectado a la nube; muestra en un mapa las coordenadas de las mediciones que pertenecen a un recorrido. Aún no se confirma su publicación como sitio estático en Render.

3. Implementación mínima y criterios de cumplimiento

Componente	Implementación verificada	Estado
Entrada física	Lectura real del acelerómetro MPU6050 y cálculo de la magnitud $\sqrt{(ax^2 + ay^2 + az^2)}$, en m/s^2.	Implementado
Embebido	ESP32 programado con ESP-IDF; se conecta por Wi-Fi y envía los datos en formato JSON por HTTPS.	Implementado
Backend nube	Servidor en Node.js/Express desplegado en Render; el POST valida los campos y el GET consulta los registros.	Implementado
Base de datos	PostgreSQL en Supabase; guarda identificador, valor, variable, unidad y fecha/hora.	Implementado
Frontend	Pantalla con la última medición, su fecha y el historial; usa los datos del servidor real.	Implementado localmente

3.1. Procesamiento y formato

Primero se calcula la magnitud de la aceleración combinando los ejes X, Y y Z. Cuando el sensor está quieto, el valor esperado es cercano a 9.81 m/s² (la gravedad), con pequeñas variaciones según su orientación y calibración. El programa toma una lectura aproximadamente cada 500 ms y hace un envío cada diez lecturas; el tiempo real entre envíos puede ser mayor porque la solicitud HTTPS también toma tiempo.

Ejemplo de solicitud POST:

{"device_id":"dashguard-001","variable":"aceleracion","valor":9.99,"unidad":"m/s2","source":"mpu6050"}

El servidor asigna el ID y la fecha y hora del registro. Si los datos son válidos responde con HTTP 201 (creado); si el JSON llega incompleto o con errores, debe responder con HTTP 400 (solicitud incorrecta).

3.2. Estructura de datos

Campo	Tipo	Uso
id	bigint / identity	Identificador único
device_id	text	Dispositivo emisor
variable	text	Nombre de la magnitud
valor	double precision	Medición numérica
unidad	text	m/s2
source	text, opcional	Procedencia: mpu6050
fecha_hora	timestamptz	Marca de tiempo del backend
created_at	timestamptz	Creación del registro

Para consultar los datos basta una solicitud GET a https://dashguard-backend.onrender.com/mediciones, que devuelve la lista de mediciones guardadas.

4. Pruebas y resultados

4.1 Prueba de registro y almacenamiento de mediciones del MPU6050 en la base de datos.

Descripción: Se comprobó que las mediciones de aceleración enviadas desde el ESP32 se guardan correctamente. El servidor desplegado en Render recibe los datos por HTTPS POST, valida la información y la registra en la base de datos PostgreSQL alojada en Supabase.

Resultado: Se encontraron diez registros guardados del dispositivo «dashguard-001», con identificadores del 1 al 10. Cada registro incluye identificador, fecha y hora de creación, dispositivo, variable medida, valor y unidad. Los valores de aceleración van de 9.77 a 11.08 m/s².

id int8	created_at timestamptz	device_id text	variable text	valor float8	unidad text
1	2026-10-13 19:04:15.943019+	dashguard-001	aceleracion	9.86	m/s2
2	2026-10-13 19:04:25.149833+	dashguard-001	aceleracion	9.97	m/s2
3	2026-10-13 19:04:34.392829	dashguard-001	aceleracion	9.96	m/s2
4	2026-10-13 19:04:43.524722	dashguard-001	aceleracion	11.08	m/s2
5	2026-10-13 19:04:52.747513+	dashguard-001	aceleracion	10.09	m/s2
6	2026-10-13 19:05:01.877945+	dashguard-001	aceleracion	9.77	m/s2
7	2026-10-13 19:36:48.606822	dashguard-001	aceleracion	10.22	m/s2
8	2026-10-13 19:36:57.776219+	dashguard-001	aceleracion	9.97	m/s2
9	2026-10-13 19:37:39.777508+	dashguard-001	aceleracion	10.17	m/s2
10	2026-10-13 19:37:48.748583+	dashguard-001	aceleracion	10.17	m/s2

Con esta prueba se comprueba que las mediciones se conservan en la base de datos y que esta tiene la estructura necesaria para almacenar y consultar la información de DashGuard.

4.2 Prueba de almacenamiento y consulta de mediciones de aceleración con coordenadas GPS.

Descripción: Se comprobó que las mediciones del acelerómetro MPU6050 se guardan junto con la ubicación del GPS NEO-6M. Los registros se consultaron desde la página de DashGuard, donde se muestran el identificador, el valor de aceleración, la unidad, la fecha, la hora, el estado del GPS, la latitud, la longitud y el número de satélites.

Resultado: Se vieron registros consecutivos del 13 de octubre de 2026, con aceleraciones entre 9.95 y 13.87 m/s². Todos tienen el estado «GPS válido», entre 7 y 8 satélites y coordenadas asociadas.

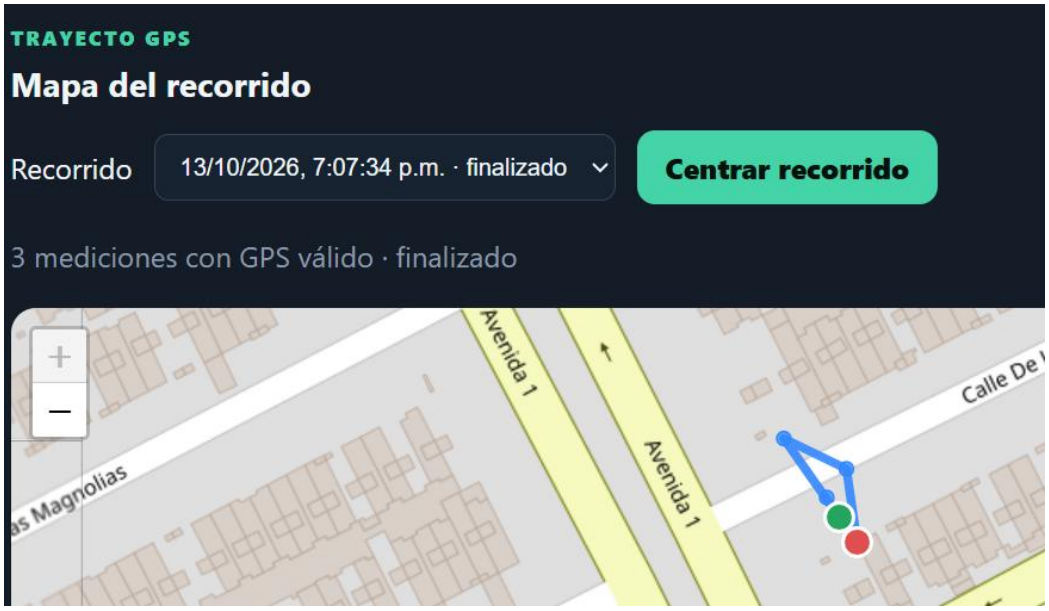
ID	VARIABLE	VALOR	UNIDAD	FECHA Y HORA	ESTADO	LATITUD	LONGITUD	SATÉLITES
#91	aceleracion	13.87	m/s2	13/10/2026, 7:08:46 p.m.	GPS valido	32.460971	-116.861805	8
#90	aceleracion	12.23	m/s2	13/10/2026, 7:08:36 p.m.	GPS valido	32.460990	-116.861788	8
#89	aceleracion	13.64	m/s2	13/10/2026, 7:08:23 p.m.	GPS valido	32.461073	-116.861800	8
#88	aceleracion	11.65	m/s2	13/10/2026, 7:07:54 p.m.	GPS valido	32.461111	-116.861895	7
#87	aceleracion	11.49	m/s2	13/10/2026, 7:07:43 p.m.	GPS valido	32.461036	-116.861829	8
#86	aceleracion	9.95	m/s2	13/10/2026, 7:07:34 p.m.	GPS valido	32.461014	-116.861816	8
#85	aceleracion	10.32	m/s2	13/10/2026, 7:07:20 p.m.	GPS valido	32.461018	-116.861818	8
#84	aceleracion	13.38	m/s2	13/10/2026, 7:07:11 p.m.	GPS valido	32.461031	-116.861841	8
#83	aceleracion	12.85	m/s2	13/10/2026, 7:06:59 p.m.	GPS valido	32.461020	-116.861852	8

Con esta prueba se demuestra que el sistema permite consultar mediciones reales de aceleración junto con su ubicación, guardadas y recuperadas a través de la arquitectura implementada.

4.3 Prueba de registro y visualización de un recorrido mediante el módulo GPS NEO-6M.

Descripción: Se hizo una prueba de seguimiento GPS con el módulo NEO-6M conectado al ESP32. Las coordenadas se enviaron al servidor y se guardaron en la base de datos para consultarlas después desde la página de DashGuard.

Resultado: En la aplicación web se puede elegir un recorrido terminado y ver su trayectoria en un mapa. En la prueba del 13 de octubre de 2026 se registró un recorrido con tres mediciones GPS válidas, dibujadas como puntos unidos que muestran la ruta, con marcadores de inicio y fin.



Con esta prueba se demuestra que la aplicación web puede consultar y mostrar gráficamente los recorridos GPS.

4.4. Problemas encontrados y correcciones

Situación	Acción / consideración
Los permisos de Supabase impedían insertar registros	Se ajustaron los privilegios de la cuenta de servicio del backend.
Error al compilar con ESP-IDF por el paquete de certificados TLS	Se añadió la dependencia mbedtls en el archivo CMakeLists y se verificó el paquete de certificados.
Wi-Fi institucional de la UABC	La conexión falló (códigos de error 2 y 5); para la demostración se recomienda usar un hotspot de 2.4 GHz con WPA2.
Diferencia entre hora UTC y hora local	La base de datos guarda la hora en formato timestamptz y la página la muestra en horario local.
Render en plan gratuito	Si la instancia estuvo inactiva, puede tardar más en responder al reactivarse; hay que tenerlo en cuenta durante las pruebas.

5. Cronograma de las semanas restantes

Semana	Actividad y entregable
1	Cerrar las evidencias de la prueba de concepto (README, mediciones y pruebas); configurar la microSD y la Raspberry Pi Zero 2 W.
2	Pasar la lectura del MPU6050 y la comunicación HTTPS a la Raspberry Pi; comparar los resultados con los del ESP32.
3	Integrar la cámara y grabar video por segmentos en la microSD.
4	Implementar la grabación circular (los videos nuevos reemplazan a los más antiguos) y proteger los clips ligados a incidentes.
5	Pasar a la Raspberry Pi el GPS NEO-6M, ya probado con el ESP32; validar la ubicación y la hora en el prototipo final.
6	Enviar a la nube los datos de los eventos; vincular los clips y consultarlos desde la web.
7	Validar la reconexión, el almacenamiento, el consumo de energía y el funcionamiento continuo.
8	Hacer pruebas integrales, corregir fallas, grabar la demostración y cerrar la documentación.

6. Presupuesto

Componente	Referencia (MXN)
Raspberry Pi Zero 2 W	$2,321.04
Cámara Raspberry Pi 1.3	$318.27
MPU6050	$132.18
GPS NEO-6M	$137.33
microSD 128 GB	$845.18
Subtotal estimado	$3,754.00

Estos precios no incluyen el UPS HAT, la batería, cables o adaptadores adicionales, la carcasa, los envíos ni posibles costos de la nube. Render y Supabase se usan en las versiones disponibles para prototipos, que tienen límites y pueden cambiar de plan.

7. Matriz de riesgos y contingencias

Riesgo	Prevención	Contingencia
Wi-Fi institucional incompatible o saturado	Probar la red antes de exponer; preparar un hotspot de 2.4 GHz.	Hacer la demostración con red móvil o con evidencias grabadas.
Render inactivo o con mucha latencia	Activar y probar las rutas del servidor antes de la demostración.	Reintentar y registrar los tiempos reales.
Pérdida de datos por desconexión	Registrar los errores y diseñar una cola local (guardar los datos hasta que haya conexión).	Reanudar el envío al reconectar; anotar los datos que se hayan perdido.
MicroSD llena o desgastada	Grabación circular, vigilar el espacio disponible y usar una memoria adecuada.	Borrar los segmentos antiguos no protegidos y cambiar la tarjeta.
Sobrecarga de la Raspberry por video y sensores	Medir el uso de CPU, RAM y temperatura; limitar la resolución.	Reducir la calidad o la frecuencia, o separar las tareas.
Falsos eventos de impacto	Recolectar muestras y ajustar los umbrales.	Revisar los eventos y recalibrar los criterios.
Alimentación insuficiente	Medir el consumo y verificar la compatibilidad con el UPS y la batería.	Usar una fuente estable durante las pruebas y rediseñar la alimentación.

8. Evaluación de viabilidad y pendientes

La prueba de concepto ya demuestra lo esencial del recorrido técnico: desde los sensores físicos (MPU6050 y GPS NEO-6M) conectados a un sistema embebido, hasta ver en la web los registros guardados en la nube. Se obtuvo una posición GPS válida y una respuesta HTTP 201 al enviar el JSON integrado. El ESP32 se usa como plataforma temporal para validar la arquitectura; la plataforma prevista para el sistema final sigue siendo la Raspberry Pi Zero 2 W.

9. Actualización: GPS y registro de recorridos

Dato GPS observado	Resultado
Latitud	32.460880°
Longitud	−116.861958°
Satélites utilizados/observados	10
Tipo de fijación	3D
Altitud	191.3 m
HDOP (precisión horizontal; mientras más bajo, mejor)	1.12

9.1 Integración embebida y transmisión

El ESP32 logró leer el MPU6050 y obtener los datos del GPS al mismo tiempo. El JSON que se envía al servidor incluye la aceleración, un indicador de GPS válido (gps_valido), la latitud, la longitud, el número de satélites y el HDOP. Durante la integración fue necesario aumentar la memoria (stack) de la tarea a 8192 bytes, y se confirmó que el servidor respondió con HTTP 201.

9.2 Registro y consulta de recorridos

Se añadieron dos botones conectados a GPIO25 y GPIO26 para marcar el inicio y el final de un recorrido. La página usa las coordenadas GPS de las mediciones de cada recorrido para mostrarlas en un mapa. Con esto la prueba de concepto crece respecto a la versión inicial, que solo mostraba una tabla de aceleración.